ProEduvate submission: copy-paste answers

Project Title

ComplaintIQ: AI-Powered Customer Complaint Classification & Resolution System

Brief Description of Your Project

ComplaintIQ reads each customer complaint and does the triage. It predicts the category, sentiment, emotion and priority, extracts order IDs, amounts, products and places, routes the complaint to a department, and drafts a reply grounded in approved playbooks for staff approval. It groups duplicate complaints, flags emerging spikes by product or city, and learns from agent corrections. Every decision comes with the reason behind it.

GitHub Repository Link

https://github.com/Aakashxnoir/AI-Powered-Customer-Complaint-Classification-Resolution-System

Live Project / Deployment Link

`<paste your Hugging Face Space URL here after deploying: see "Deploy" at the end of this file>`

Project Documentation Link

https://github.com/Aakashxnoir/AI-Powered-Customer-Complaint-Classification-Resolution-System/blob/main/docs/DOCUMENTATION.md

Upload Project Files (ZIP/PDF)

`ComplaintIQ-submission.zip` (source, documentation, evaluation artifacts; under 10 MB).

Technologies / Tools Used

Python 3.12, FastAPI, SQLAlchemy, PostgreSQL (SQLite fallback), PyTorch (CPU), Hugging Face Transformers, Sentence Transformers (all-MiniLM-L6-v2), scikit-learn, spaCy, pandas, NumPy, Google Gemini API (retrieval-augmented drafting), React 19, Vite, Tailwind CSS, Recharts, pytest, Docker, uv, Git/GitHub.

Explain your approach to solving the assigned problem statement

I treated the problem as one pipeline where every step returns a result and the reason for it, so a support agent can trust or correct it.

1. **Data.** No public labelled dataset of angry complaints exists. I combined the Bitext customer-support dataset (real-style requests, 5 of 6 categories) with 72 synthetic templates (12 per category) filled with order IDs, products, amounts and cities. The synthetic split is by template, so the test set holds sentence patterns the model never saw.
2. **Classification.** One 384-d MiniLM sentence embedding plus logistic regression. It beat a TF-IDF baseline by 11 accuracy points (92.7% vs 81.3%) and retrains in seconds from cached embeddings. The same embedding also powers duplicate detection, similar-complaint search and playbook retrieval.
3. **Sentiment and emotion.** Pretrained RoBERTa models, with an escalation to "highly negative" from intensity markers.
4. **Entities.** Regex for fixed formats (order ID, amount, email, phone), a catalogue gazetteer for products and cities, spaCy for dates and other places.
5. **Priority, urgency, severity and routing.** Deliberately rules, not ML: they are business decisions that must be auditable and editable. ML signals (sentiment, frustration, entities, duplicates) feed the rules, and every point of the score is listed with its reason.
6. **Duplicates.** A duplicate is the same case raised again: high similarity plus a shared customer or order ID. Near-identical wording from different customers is a trend, not a duplicate.
7. **Resolution.** Retrieve the closest approved playbooks, have Gemini draft a reply grounded only in them, drop citations outside the retrieved set, and fall back to a template if the call fails. Staff must approve, edit or reject; nothing is sent automatically.
8. **Learning.** Agents can correct category, sentiment, priority and department. A guarded retrain deploys only if accuracy stays within 1 point of the original model.

What challenges did you face while completing the project?

- **No real labelled data.** I had to build a dataset from Bitext plus synthetic templates, then split by template to avoid train/test leakage. I report the remaining weaknesses: template data overstates entity F1, 53 of 457 Bitext test rows are near-duplicates of training rows, and the sentiment score is partly circular.
- **Duplicate detection precision.** Plain cosine similarity flagged hundreds of different-customer complaints as duplicates (precision about 0.2). Adding identity rules (same customer or order ID) raised precision to 0.93 at recall 1.00.
- **Explainability that is faithful.** I used word-occlusion on the real embedding model rather than a proxy model, and capped it to the first 80 words to bound latency.
- **Safe LLM use.** Drafts could cite playbooks that were never retrieved or follow instructions hidden in a complaint. I pass the complaint as delimited data, drop unretrieved citations, and keep a template fallback.
- **Safe retraining.** Repeated retrains could drift down a point at a time, so the gate compares against the original model, not the last one.
- **Deployment.** Three transformer models need several GB of RAM, so the free tiers of most hosts do not fit. I packaged one Docker image that bakes in the models and serves the frontend and API from one origin.
- **Gemini API changes.** Model names were retired mid-project; failures were silent because of the template fallback, so I verified live generation explicitly.

What features or functionalities have you implemented?

- Complaint intake by form and CSV upload (per-row error report, size and row limits, HTML/URL stripping, exact-duplicate-row skip).
- Category classification (6 classes) with per-class confidence and highlighted important words.
- 4-level sentiment, 7-emotion scores and a frustration score.
- Entity extraction: order ID, amount, email, phone, product, city, dates.
- Priority with urgency and severity scores and an itemised reason list; fraud and safety overrides.
- Department routing with a routing reason.
- Duplicate detection with human confirm/reject, and a top-5 similar-complaints view.
- Emerging-issue detection by category, product and location (z-score over a 14-day baseline).
- RAG reply drafting with Gemini over 14 approved playbooks, cited sources, template fallback, and staff approve/edit/reject.
- Human feedback loop, per-field agreement, weak-spot report, and guarded retraining.
- Evaluation dashboard: confusion matrix, baseline comparison, per-class F1, duplicate and trend metrics.
- Landing page and a 7-page React console; 22 automated tests; Docker deployment.
- Safety guards: host allow-list, cross-origin write refusal, anti-framing headers.

Is there anything else you would like the evaluation team to know?

- The hosted instance starts with an empty database; there is no mock data in the app. Submit a complaint on the **New complaint** page or upload a CSV with a `text` column to see the full pipeline.
- Without a `GEMINI_API_KEY` the app runs in template-fallback mode; the header shows which mode is active.
- Reported metrics are measured, with caveats listed in section 7 of the documentation: synthetic data overstates real-world performance, sentiment labels are weak, priority weights are hand-set, and there is no authentication.
- A narrated walkthrough video is in the repository under `demo/` (not in the ZIP because of the 10 MB limit).
- Reproduce everything: `ml/generate_data.py`, `ml/train.py`, `ml/evaluate.py`, then `uv run pytest -q` (22 tests).

Candidate Declaration

Tick all three boxes yourself. The form asks you to confirm the work is your own and the links are accurate; I cannot confirm that for you.

Deploy (Hugging Face Spaces, free, 16 GB RAM)

1. Create a Space at huggingface.co/new-space: SDK **Docker**, hardware **CPU basic**.
2. Add this block as the first lines of a `README.md` in the Space repo (keep the rest of the repo as is): ```

---

title: ComplaintIQ sdk: docker app_port: 7860

`` 3. Push the repo to the Space remote:git remote add space https://huggingface.co/spaces//complaintiq && git push space main. 4. Optional: in Space Settings → Secrets addGEMINI_API_KEY. The first build takes about 10 minutes. 5. Paste the Space URL (https://-complaintiq.hf.space`) into the Live Project field.